Interactive data visualizations for US communities crime exploration

Fabio Dollaku, HPDAV Individual Assignment, 2026

1. Problem, Data, and User Tasks

I built this for the US Communities and Crime dataset from the UCI ML Repository. It has 1,355 communities across 46 states and over 100 normalized attributes covering things like income, housing density, policing, and crime rates. Some values are missing in the original data (marked as "?"), so I filter those out before doing anything with them.

The main things I wanted to support: comparing any two continuous attributes side by side, seeing which states certain communities group into, being able to brush-select a subset and track it across both views, and drilling into a state to see its individual communities. Stack is React 18 + D3.js 7 + Redux Toolkit.

2. Visual Design and Justification

Two panels side by side: scatterplot on the left, hierarchy on the right.

The scatterplot puts two user-chosen attributes on x and y. You pick them from dropdowns and can swap them anytime. I added a D3 brush so you can drag a rectangle to select a group of points; selected ones go to full opacity with a red stroke, the rest fade to 0.3. The idea is that once you pick out an interesting cluster, you can immediately see where those communities sit in the hierarchy.

For the hierarchy, communities sit under their state (two-level: state then community). I implemented three layouts and compared them:

Layout	What works	What doesn't
Treemap	Uses space well; area and color both encode the value, so you get two channels at once	The hierarchy itself is less obvious
Circle packing	Clear grouping; looks nice; containment is intuitive	Wastes a lot of space between circles
Tree (node-link)	You can actually see the parent-child structure clearly	Bad for comparing values; spreads out too wide

Treemap: each community is a rectangle whose area is proportional to its size in the hierarchy. Color (yellow to red) encodes the chosen attribute value, so you get two visual channels at once. States have labeled borders that make the geographic grouping obvious. The downside is that very small communities get tiny cells that are hard to click. This is my preferred layout because the area + color combination makes it easy to spot high-crime states at a glance.

Circle packing: communities are circles packed inside larger state circles. The layout is visually engaging and makes state boundaries clear, but the wasted whitespace between circles makes it harder to compare sizes accurately. Clicking small community circles is also fiddly. Good for getting a quick sense of which states have the most communities.

Tree layout (horizontal dendrogram): states branch off a root node, with communities as leaves. Every node is the same size so there is no area encoding, just topology. It is the clearest way to see the hierarchy structure itself, but with 1,355 communities the leaf nodes overlap badly and color is the only way to distinguish values. Useful for understanding structure, not so great for spotting patterns in attribute values.

I made treemap the default because it encodes two things at once: cell area and color. I used a yellow-to-red scale for the chosen attribute, so dark red cells stand out immediately and you can spot the high-value communities in each state pretty fast. States are separated by gray borders with a label at the top. The other layouts are still there in the dropdown if you want a different view.

Figure 1: Treemap layout overview

Figure 2: Brush selection with synchronized highlighting

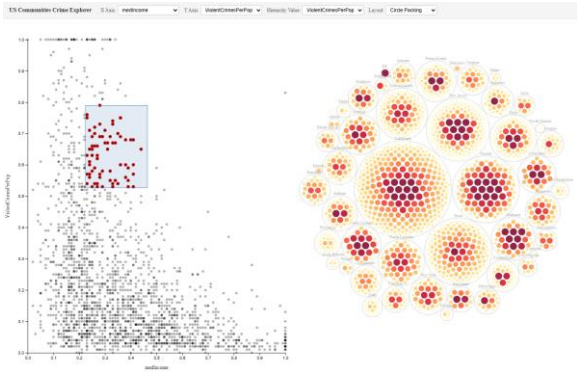

Figure 3: Circle packing layout

Figure 4: Tree (node-link) layout

3. Synchronized Interactions

The two views share state through Redux. When you brush the scatterplot, it dispatches the selected indices to the store; the hierarchy reads that and highlights the matching nodes, fading everything else. Same thing in reverse: clicking a node in the hierarchy dispatches a selection and the scatterplot highlights those points in red.

In practice you can do things like brush a cluster of high-crime outliers and immediately see which states they come from, or click a state in the treemap and see where its communities land in attribute space. The X/Y axes and the hierarchy value attribute are all controlled independently, so you can explore combinations without losing your current selection. I added transitions (300-1000ms depending on what changed) so it does not feel too jumpy.

Brush selection on scatterplot synced to treemap hierarchy

Declaration of Generative AI Use

This is my own work. I used Claude during development mainly to debug. When D3 rendering was off or Redux state was not syncing right, I used it to figure out what was going wrong. I also used it to look up D3 API stuff like how brush events work or the different hierarchy layout APIs. The design choices (coordinated views, treemap as default, the opacity-based highlighting) came from the course material. Claude did not write this report.